Part 1: Recap of reference triangle exercise

We want to approx. an integral on an arbitrary triangle T: $\int_T f(x)\,dx$

Last week we introduced $T_{ref} = \{(x,y): 0 \le x \le 1,\ 0 \le y \le 1-x\}$, $|T_{ref}| = \frac{1}{2}$

We said we wanted this quadrature rule to be reusable on different triangles, instead of constructing a rule for every other triangle.

Step 1 - qrule = QRule (method, f) ⇝ calls class constructor _init_

This will select a reference quadrature rule

Depending on "method" one of the predefined formulas is selected!

→ p1o2, p3o2, p3o3, p7o6 are the methods for which ref. nodes and weights have already been calculated.

→ For GaussDuffy2 and GaussDuffy3 we will implement buildDuffyqf

At the end of _init_:

self.nodes : ref. nodes
self.weights: ref. weight

Todo (a)

Let the physical triangle have vertices V_0, V_1, V_2, we know the vertice of T_{ref} is $(0,0), (1,0), (0,1)$

Transformation: $x = V_0 + A\hat{x}$, $A = \begin{bmatrix} V_1 - V_0 & V_2 - V_0 \end{bmatrix}$

$(0,0)$: $V_0 + 0 \cdot (V_1 - V_0) + 0 \cdot (V_2 - V_0) = V_0$

$(0,1)$: $V_0 + 0\,(V_1 - V_0) + 1 \cdot (V_2 - V_0) = V_0 + V_2 - V_0 = V_2$

$(1,0)$: $V_0 + 1\,(V_1 - V_0) + 0 \cdot (V_2 - V_0) = V_0 + V_1 - V_0 = V_1$

```
A = np.array([
        Tri[1, :2] - Tri[0, :2],
        Tri[2, :2] - Tri[0, :2]]).T
```

arranges them as rows

but we want columns

Slicing [row, column]

⇒ [1, :2] first row, take first 2 columns (index 0 and 1)

Now we have to transform all quadrature nodes!

$c_i = V_0 + A\hat{c}_i \rightsquigarrow \hat{c}_i$ is one node $\Rightarrow \hat{C} \in \mathbb{R}^2$ → self. nodes

Trace the dimensions: $A \in \mathbb{R}^{2\times 2}$, $\hat{C} \in \mathbb{R}^{2\times n} \Rightarrow A\hat{C} \in \mathbb{R}^{2\times n}$

$V_0 = \text{Tri}[0, :2] \Rightarrow \text{Tri}[0, :2].\text{shape} = (2,)$

because numpy doesn't treat vectors as matrices automatically!

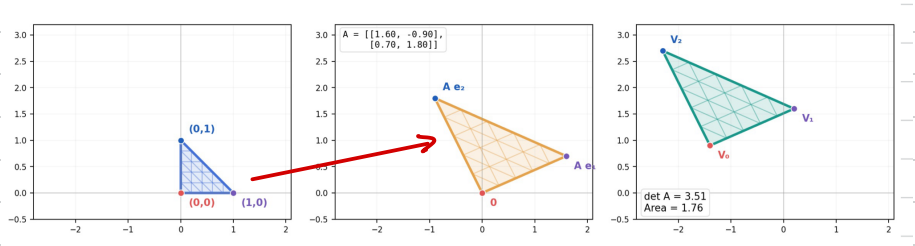
numpy broadcast

```
nodes = Tri[0, :2].reshape((2,1)) + A @ self.nodes
```

Also account for change of Area:

$$\int_T f(x)\,dx = |\det A| \int_{T_{ref}} f(V_0 + A\hat{x})\,d\hat{x} \approx |\det A| \sum_i w_i f(c_i)$$

unnormalized weights



One detail about the particular implementation is that $\sum_i \text{self.weights} = 1$, so they are normalized: $= \hat{w}_i$

Consider $f(\hat{x}) = 1 \rightarrow \int_{T_{ref}} 1\,d\hat{x} = |T_{ref}| = 1/2$

Our quadrature gives:

$$\sum_{i=1}^{\hat{n}} w_i f(\hat{c}_i) = \sum_{i=1}^{\hat{n}} w_i$$

If quadrature is exact for constant functions we require: $\sum_i w_i = 1/2$

But rather than storing them like this we normalize:

$$\hat{w}_i = 2w_i \Rightarrow \sum_i \hat{w}_i = 2\sum w_i = 2 \cdot 1/2 = 1$$

It holds that $w_i = \frac{1}{2}\hat{w}_i$

$$\Rightarrow \int_{T_{ref}} f(\hat{x})\,\hat{x} \approx \sum_i w_i f(\hat{c}_i) = \sum_i \frac{1}{2}\hat{w}_i f(\hat{c}_i)$$

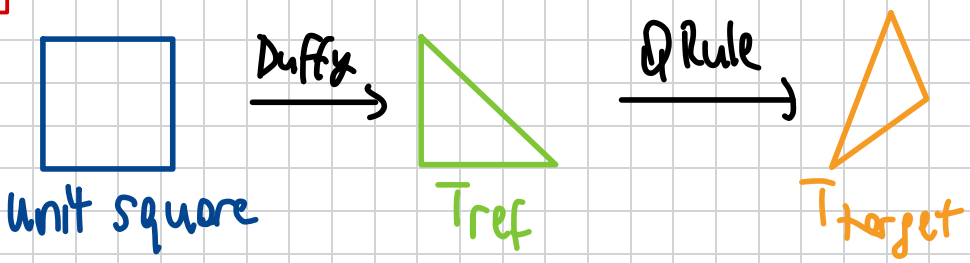
$$\Rightarrow Q_T(f) = \frac{|\det A|}{2} \sum_i \hat{w}_i f(V_0 + A\hat{c}_i) \ldots \text{we return the result!}$$

Todo (b) tryorder

We test monomials $x^{k_1} y^{k_2}$ for $k = k_1 + k_2$!

```
pol = lambda x: (x[0]**k1) * (x[1]**k2)
```

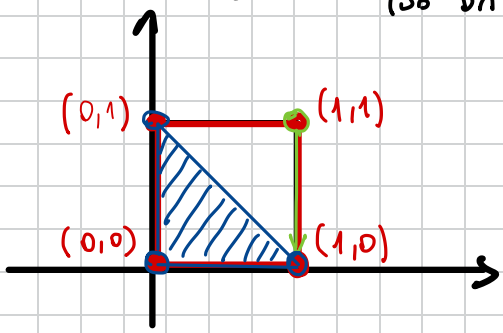
Todo (c)



Q Rule takes a quadrature rule (weight / nodes) on Tref and transforms it to any other triangle.

Take one step back → How do we get the reusable quadrature rule? (so on Tref)

$\Rightarrow (u,v) \in [0,1]^2 \Rightarrow$



$\Rightarrow$ Let $x = u$, $y = v(1-u)$

$x = 0$ $\quad y = v \Rightarrow 0 \le v \le 1$

$x = 1$ $\quad y = 0$

$\Rightarrow$ Observe that there is an area change again!

generate nodes for both direction by roots_legendre

$$J = \begin{bmatrix} \partial x/\partial u & \partial x/\partial v \\ \partial y/\partial u & \partial y/\partial v \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -v & 1-u \end{bmatrix} \Rightarrow \det J = 1-u \quad \dots \text{ depends on } u!$$

$$\Rightarrow dx\,dy = |\det J|\, du\, dv$$

$$\Rightarrow I = \int_{T_{ref}} f(x,z)\, dx\, dy = \int_0^1 \int_0^1 f(u, v(1-u))\, |\det J|\, dv\, du$$

For nodes we know $\Rightarrow \hat{c}_{ij} = \begin{pmatrix} u_i \\ v_j(1-u_i) \end{pmatrix}$ where u_i, v_j belong to c_{ij} on unit square

- Remember 2D Quadrature: We said weight of the 2 dimensions get multiplied to create the weights for quad rule.

$$\Rightarrow w_{ij} = w_i^{(u)}\, w_j^{(v)}\, |\det J|$$

Again we have to account for that area change!

We return weight /2 ?

→ We generated our nodes by roots_legendre for both directions, and this gives nodes on $[-1,1]$. But we are operating on $[0,1]$.

For a second just think about ordinary gauss-legendre:

$$\int_{-1}^{1} f(\tau)\, d\tau \approx \sum_{i=1}^{n} b_i f(\tau_i)$$ → b_i, τ_i is what roots_legendre return

Test $f(\tau)=1$: $\int_{-1}^{1} 1\, d\tau = 2 \Rightarrow \sum_{i=1}^{n} b_i = 2 \quad (1)$

Now transform $[-1,1]$ to $[0,1]$: $u = \frac{1+\tau}{2} \Rightarrow u \in [0,1]$

$$\int_0^1 f(u)\, du \quad \text{substitute} \quad u = \frac{1+\tau}{2} \Rightarrow du = \frac{1}{2} d\tau$$

$u=0 \to \tau=-1$; $u=1 \to \tau=1$

$$\Rightarrow \int_{-1}^{1} f\left(\frac{1+\tau}{2}\right) \frac{1}{2} d\tau = \frac{1}{2} \int_{-1}^{1} f\left(\frac{1+\tau}{2}\right) d\tau$$

$$(A) \approx \frac{1}{2} \sum \underline{b_i} f\left(\frac{1+\tau_i}{2}\right)$$ → again original nodes and weights from roots_legendre

this holds for all dims.

$$I = \int_{T_{ref}} f(x_1, x_2)\, dx\, dy = \int_0^1 \int_0^1 f(u, v(1-u)) \, |\det J|\, dv\, du$$

Let $G(u,v) = f(u, v(1-u)) \cdot |\det J|$

$$\Rightarrow I = \int_0^1 \int_0^1 G(u,v)\, du\, dv \quad \text{we know} \quad \int_0^1 G(u,v)\, dv \overset{(A)}{\approx} \frac{1}{2} \sum b_j^{(v)} G(u, v_j)$$

$$\Rightarrow I \approx \int_0^1 \left[\frac{1}{2} \sum_j b_j^{(v)} G(u, v_j)\right] du \text{ , apply same thing again}$$

$$\Rightarrow I \approx \frac{1}{4} \sum_i \sum_j b_i^{(u)} b_j^{(v)} G(u_i, v_j) = \frac{1}{4} \sum_i \sum_j b_i^{(u)} b_j^{(v)} f(u_i, v_j(1-u_i)) \, |\det J|$$

So our triangle weights are (not yet normalized)

$$W_{ij} = \frac{1}{4} b_i^{(u)} b_j^{(v)} |\det J| \quad \text{with } |\det J| \to (1-u_i)$$

As these must exactly integrate the area of T_{ref} we have

$$\sum_{ij} W_{ij} = \frac{1}{2}$$

But we said QRule wants weights that sum to 1.

$\Rightarrow$ well then $\hat{w}_{ij} = 2 W_{ij} \Rightarrow \sum_{ij} W_{ij} = \sum_{ij} \frac{1}{2} \hat{w}_{ij} = \frac{1}{2} \Rightarrow \sum_{ij} w_{ij} = 1$

and $\hat{w}_{ij} = 2 \cdot \left[\frac{1}{4} b_i^{(u)} b_j^{(v)} (1-u_i) \right] = \frac{1}{2} b_i^{(u)} b_j^{(v)} (1-u_i)$

```
k = 0
for c,b in zip(cn, bn):
    for x,w in zip(cp, bp):
        nodes[k] = [c, x*(1-c)]
        weights[k] = b*w*(1-c) # shold be /2
        k += 1

return nodes, weights/2 # here is the missing 1/2
```

c_n, b_n ... nodes/weights in u dimension
c_p, b_p ... nodes/weights in v dimension

$\Rightarrow \hat{c}_{ij} = (u_i, v_j (1-u_i))$

$\Rightarrow W_{ij} = b_i^{(u)} b_j^{(v)} (1-u_i)$

$\hat{w}_{ij}$

Part 2: Completing Quadrature

Recall for a composite quadrature rule of order p, the global error behaves like

$$|E(h)| = O(h^p)$$

Now we will see a method like Richardson-Formula, but for quadrature

→ Idea is same, we calculate integral once with width h, once with h/2 and combine them to cancel some of their errors!

Romberg Quadrature

Let T be the composite trapezoidal rule, we know it has order 2.

$$\Rightarrow T(h) - I = O(h^2) = c_1 h^2 + c_2 h^4 + c_3 h^6 + \dots$$

↑ Only even because of Taylor expansion cancels odd powers one step before this. (NOT IMPORTANT)

Derivation: Romberg Step

$$T(h) = I + c_1 h^2 + c_2 h^4 + c_3 h^6 + \dots$$

$$T(h/2) = I + c_1 \frac{h^2}{4} + c_2 \frac{h^4}{16} + c_3 \frac{h^6}{64} + \dots$$

$$\rightarrow 4T(h/2) - T(h) = 3I + \underbrace{c_1 h^2 - c_1 h^2}_{=0} + O(h^4)$$

→ Divide by 3:

$$T_1(h) = \frac{4T(h/2) - T(h)}{3} = I + O(h^4)$$

Extra: Connection to Simpson Rule

On $[a,b]$ we know $h = b-a$, $m = \frac{a+b}{2}$

One Trapezoid gives $T(h) = \frac{h}{2}(f(a)+f(b))$



Two smaller trapezoids gives: $[a,m]$, $[m,b]$

$$\Rightarrow \underbrace{\frac{h'}{2}(f(a)+f(m))}_{h' = m-a = \frac{b-a}{2} = \frac{h}{2}} \text{ and } \frac{h'}{2}(f(m)+f(b)) \Rightarrow \frac{h}{4}[f(a)+2f(m)+f(b)]$$

$$\Rightarrow \frac{4T(h|2) - T(h)}{3} = \frac{h}{6}[f(a) + 4f(m) + f(b)] \quad \text{... exactly Simpson Rule}$$

General Recurrence

$$R_{\ell,0} = T(h|2^{\ell})$$

So first column is for $T(h), T(h|2), T(h|4), \dots$. They are comp. trapezoidal on progressively finer grids.

For k, ℓ:

$$R_{\ell,k} = \frac{4^k R_{\ell,k-1} - R_{\ell-1,k-1}}{4^k - 1}$$

Other ways to improve quadrature

- Newton-Cotes
- Clenshaw-Curtis:

We select the nodes using Chebyshev points:

$$x_j = \cos\left(\frac{j\pi}{n}\right), \quad j = 0, \dots, n$$

Remember these cluster at around the enpoints.

→ Refer to script for the code for it!

Part 3: Monte Carlo Quadrature

Consider $I = \int_0^1 f(x)\,dx$, in classical deterministic quadrature we choose spesific nodes and combine the function values with weights.

Monte Carlo chooses random points;

$$t_1, t_2, \dots, t_N \in [0,1]$$

If they are uniformly distributed, we approximate

$$I_N = \frac{1}{N} \sum_{i=1}^{N} f(t_i)$$

Example – $f(x) = x^2$

$$\int_0^1 x^2\,dx = \frac{1}{3}$$, we select $t_1 = 0.1$ $t_2 = 0.4$ $t_3 = 0.6$ $t_4 = 0.7$

$\rightarrow f(t_1) = 0.01$, $f(t_2) = 0.16$, $f(t_3) = 0.36$, $f(t_4) = 0.49$

$\rightarrow I_4 = \frac{1.02}{4} = 0.255$, not that close to $\frac{1}{3}$

Important: MC does not guarantee a good result with a small number of samples. It becomes statistically reliable as the number of indep. samples grow.

Monte Carlo on a general interval

- First generate $t_i \sim U(0,1)$, so t_i uniformly distr. on $[0,1]$
- Then use $s_i = a + (b-a)t_i$ to transform them to $[a,b]$
- $I_N = \frac{b-a}{N} \sum_{i=1}^{N} f(s_i)$

Probability Course

assigns a numerical value to each outcome of a random exp.

A random variable X is the outcome of a random experiment

For MC we use a continuous rv $X \sim U(a,b)$

Its density function is $p(x) \begin{cases} \frac{1}{b-a}, & a \le x \le b \\ 0, & \text{otherwise} \end{cases}$

Intuition: Uniform sampling means equally long parts of the intervals are equally likely to contain a random point.

Expected value $\mathbb{E}(\dots)$ for a r.v. X with p

$\Rightarrow \mathbb{E}(f(X)) = \int_a^b f(x)p(x)\,dx$ for uniform $= \frac{1}{b-a}\boxed{\int_a^b f(x)\,dx} = I$

$\Rightarrow$ Therefore $I = (b-a)\,\mathbb{E}(f(X))$

↳ by estimating the expected value for the function under a uniform r.v. we can calculate the integral

Now general MC $\Rightarrow I_N = \frac{b-a}{N}\sum_{i=1}^{N} f(X_i)$ is an **unbiased** estimator

↳ $X_i \overset{iid}{\sim} U(a,b)$

$\Rightarrow \mathbb{E}(I_N) = \mathbb{E}\left(\frac{b-a}{N}\sum_{i=1}^{N} f(X_i)\right) = \frac{b-a}{N}\sum_{i=1}^{N}\mathbb{E}(f(X_i))$ → $\forall i,j: \mathbb{E}(f(X_i)) = \mathbb{E}(f(X_j))$

$= \frac{b-a}{N} N\,\mathbb{E}(f(X)) = (b-a)\,\mathbb{E}(f(X)) = I$

$\Rightarrow$ Hence unbiased!

Intuitively unbiasedness means that repeating the MC experiment many times and taking the average of them will approach to the true integral, although some experiments overestimate or underestimate I.

Variance ... measures the spread around the mean

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}(X))^2] = \mathbb{E}(X^2) - (\mathbb{E}(X))^2$$

Standard deviation ... $\sigma_X = \sqrt{\mathrm{Var}(X)}$

↳ tells the spread in the unit of the original data

Variance affects MC

If the random function values fluctuates strongly the est. integral does, too.

Example:

Consider two functions on $[0,1]$:

$$f_1(x) = 1 \quad , \quad f_2(x) = 2x$$

Their integrals are both 1:

$$\int_0^1 f_1(x)\,dx = 1 \qquad \int_0^1 f_2(x)\,dx = 1$$

$f_1(x) = 1$, means regardless of the sample f. value is 1. Therefore $Var(f_1(X)) = 0$.

For $f_2(x) = 2x$ where $X \sim U(0,1) \Rightarrow 2X \sim U(0,2)$

$$Var(2X) = 4(\mathbb{E}(X^2) - \mathbb{E}(X)^2) = 4\left(\int_0^1 x^2\,dx - \left(\int_0^1 x\,dx\right)^2\right)$$

↳ $\mathbb{E}(g(X))$ where $g(X) = x^2$

$$= 4\left(\frac{1}{3} - \left(\frac{1}{2}\right)^2\right) = \frac{4}{3} - 1 = \frac{1}{3}$$

$Var(2X) = 1/3$

Why does MC converge like $N^{-1/2}$?

$$Var(I_N) = Var\left(\frac{b-a}{N}\sum_{i=1}^{N} f(X_i)\right)$$

$Var(cX) = c^2 Var(X)$

$$= \left(\frac{b-a}{N}\right)^2 \sum_{i=1}^{N} Var(f(X_i)) = \frac{(b-a)^2}{N} Var(f(X))$$

$X_i \overset{iid}{\sim} U(a,b)$

⇒ Take the square root:

$$\sigma_{I_N} = \frac{(b-a)\,\sigma_f}{\sqrt{N}} \quad \text{where} \quad \sigma_f = \sqrt{Var(f(X))}$$

⇒ So the typical statistical error depends on two factor:

$$\frac{1}{\sqrt{N}} \times \underbrace{(b-a)\,\sigma_f}_{\text{variation of sampled vals}}$$

↓ sample count

Increasing N is expensive: $\frac{1}{\sqrt{N'}} = \frac{1}{2} \cdot \frac{1}{\sqrt{N}} \Rightarrow N' = 4N$

⇒ Reducing the error by a factor of 2, we need 4 times as many samples

⇒ Reason that we rely on variance reduction methods!

Part IV: Variance Reduction

Look at the second factor in the error: $(b-a)\,\sigma_f$ ← variation of sampled vals

⇒ Decreasing σ_f also helps.

Control Variates

As usual we want to calculate $I = \int_a^b f(x)\,dx$

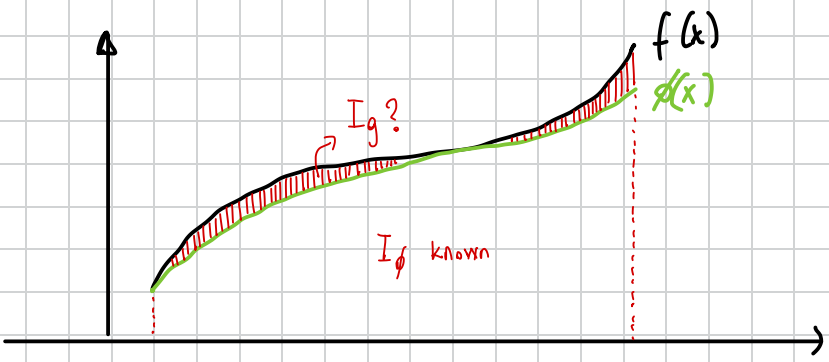
Assume we can approximate f with ϕ that is simple, approximates well and most importantly we know the integral of ϕ exactly.

$$f(x) = \phi(x) + [f(x) - \phi(x)], \text{ trivially.}$$

⇒ Integrate both sides:

$$I = \int_a^b f(x)\,dx = \underbrace{\int_a^b \phi(x)\,dx}_{I_\phi \text{ known}} + \underbrace{\int_a^b [f(x) - \phi(x)]\,dx}_{\text{Approx. using MC}}$$

As ϕ approximates f well, know we are applying MC on $g(x) = f(x) - \phi(x)$ which has much less variation as ϕ follows f closely.



Does the error really decrease?

The control-variate estimator uses:

$$I_N^{CV} = I_\phi + \frac{b-a}{N}\sum_{i=1}^{N}[f(X_i) - \phi(X_i)]$$

$$\begin{aligned}
Var\left(I_N^{CV}\right) &= Var\left(I_\phi + \frac{b-a}{N}\sum_{i=1}^{N}[f(X_i) - \phi(X_i)]\right) \\
&= Var\left(\frac{b-a}{N}\sum_{i=1}^{N}[f(X_i) - \phi(X_i)]\right) && Var(A+c) = Var(A) \\
&= \left(\frac{b-a}{N}\right)^2 Var\left(\sum_{i=1}^{N}[f(X_i) - \phi(X_i)]\right) \\
&= \left(\frac{b-a}{N}\right)^2 \sum_{i=1}^{N} Var(f(X_i) - \phi(X_i)) && X_i \text{ indep} \\
&= \frac{(b-a)^2}{N} Var(f(X) - \phi(X))
\end{aligned}$$

$\Rightarrow Var(I_1) = \frac{(b-a)^2}{N} Var(f(X))$ earlier, so we have $Var(I_N^{CV}) < Var(I_N)$ if $Var(f(X) - \phi(X)) < Var(f(X))$

Why is a Taylor polynomial a good control variate?

It is smooth and can be integrated exactly.

Reminder - Taylor poly of degree 3 around some x_0

$$\phi_3(x) = f(x_0) + f'(x_0)(x-x_0) + \frac{f''(x_0)}{2!}(x-x_0)^2 + \frac{f'''(x_0)}{3!}(x-x_0)^3$$

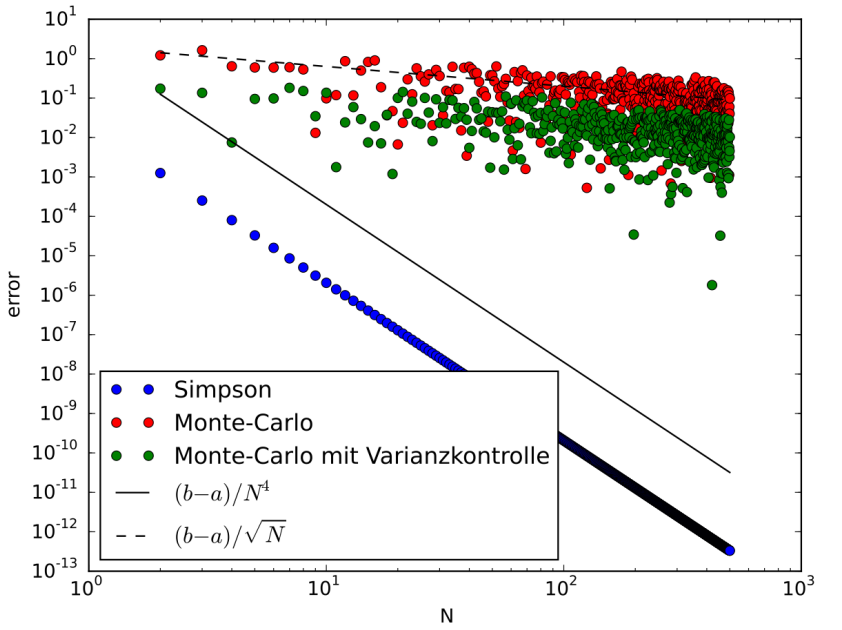
$f(x) - \phi_3(x) = \mathcal{O}((x-x_0)^4)$ as our example Taylor poly. can capture up to first three derivatives, error comes afterwards.

→ So here that remaining error term is important. If this fluctuates less than original function, MC is more accurate.

However Taylor poly is local. On a long interval, it may become less accurate far from the expansion point

Code Expert: Einfaches MC - Vergleich

Reproduce →



We compare three methods on:

$$I = \int_0^2 f(x)\,dx \qquad f(x) = \frac{e^x - 1}{e - 1}$$

Notation:

rng -- random number generator
phi ... cubic Taylor control variate
fminusphi ... $f - \phi$

Iexact ... exact integral of f
Iphiexact ... exact integral of ϕ

Coding Tips

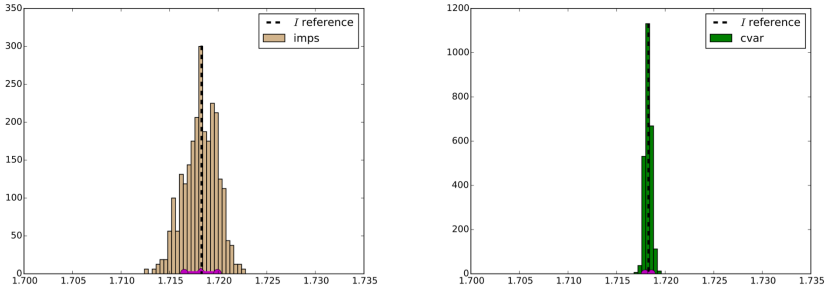
- my can generate values in $[0,1)$ but we need them in $[a,b]$, transform them!
- For Simpson, N is the number of panels. For MC, N is the number of random samples.
- Define phi and Iphiexact (integrate by hand)
- For I_N^{cv} you must add I_p, don't forget!

Discussion

For Simpson, we know it is deterministic.

For MC and MC+CV, each point is an experiment. So a different experiment can produce (even if N increased) a temporarily worse approx.

Variance reduction →



Two extensions

- Importance sampling
- Quasi – Monte Carlo

Sparse grids

↗ $O(n^d)$

